

In JavaScript, the `this` keyword refers to the execution context object where the current code is running. Its value is dynamically determined at runtime based on *how* a function is called, not where it is defined.

```
function Person(name) {  
  this.name = name;  
}  
  
var p = new Person("Ved");
```

In JavaScript, every constructor function has a prototype object. Methods placed on the prototype are shared by all objects created using that constructor.

prototype is a property of functions in JavaScript. It is used to share methods and properties among all objects created by a constructor function.

`const p = {};` // Step 1: Create a new empty object

`p.proto = Person.prototype;` // Step 2: Link the prototype

`Person.call(p, "Ved");` // Step 3: Call the function with `this = p`

// Inside Person:

`this.name = "Ved";` // Equivalent to `p.name = "Ved"`

`return p;` // Step 4: Return the object



```
function Person(name) {  
    this.name = name;  
}  
  
// Prototype method  
Person.prototype.greet = function ()  
{  
    console.log("Hello, I am " +  
this.name);  
};  
  
const p1 = new Person("Ved");  
const p2 = new Person("Rahul");  
  
p1.greet(); // Hello, I am Ved  
p2.greet(); // Hello, I am Rahul
```